Experiment 8: Kuhn-Tucker Conditions - Lagrangian Multiplier Method

Problem 1

Aim: Minimize $f(x, y) = x^2 + y^2$ subject to $x + y \geq 2$, $x \geq 0$, $y \geq 0$.

Procedure: Form Lagrangian $\mathcal{L} = x^2 + y^2 - \lambda(x + y - 2) - \mu_1 x - \mu_2 y$. Solve $\nabla\mathcal{L}$=0: $2x = \lambda + \mu_1$, $2y = \lambda + \mu_2$; primal/dual feasibility; complementary slackness.

CODING:

```
[1]
0s
from scipy.optimize import minimize

def obj(x):
    return x[0]**2 + x[1]**2

cons = [
    {'type': 'ineq', 'fun': lambda x: x[0] + x[1] - 2},
    {'type': 'ineq', 'fun': lambda x: x[0]},
    {'type': 'ineq', 'fun': lambda x: x[1]}
]

res = minimize(obj, [1, 1], constraints=cons)

print(f"Optimal: {res.x}, Value: {res.fun}")

Optimal: [1. 1.], Value: 2.0
```

OUTPUT:

Optimal: [1. 1.], Value: 2.0

Problem 2

Aim: Maximize $f(x, y) = xy$ subject to $x + y \leq 100$, $x \leq 40$, $x, y \geq 0$.

Procedure: Lagrangian $\mathcal{L} = xy + \lambda_1(100 - x - y) + \lambda_2(40 - x) + \mu_1 x + \mu_2 y$. Stationarity conditions and slackness.

Coding:

```
[2]
✓ 0s
def obj(x): return -(x[0]*x[1])
cons = [{'type':'ineq', 'fun': lambda x: 100-x[0]-x[1]},
        {'type':'ineq', 'fun': lambda x: 40-x[0]},
        {'type':'ineq', 'fun': lambda x: x[0]},
        {'type':'ineq', 'fun': lambda x: x[1]}]
res = minimize(obj, [20,20], constraints=cons)
print(f"Maximum: {-res.fun}, Point: {res.x}")

••• Maximum: 2399.9999999302554, Point: [40. 60.]
```

OUTPUT:

Maximum: 2399.9999999302554, Point: [40. 60.]

Problem 3

Aim: Minimize $f(x, y) = (x-2)^2 + (y-3)^2$ subject to $x + 2y \geq 5$, $x, y \geq 0$.

Procedure: $\mathcal{L} = (x-2)^2 + (y-3)^2 - \lambda(x + 2y - 5) - \mu_1 x - \mu_2 y$. Solve KKT system.

Coding:

```
[3]
✓ 0s
def obj(x): return (x[0]-2)**2 + (x[1]-3)**2
cons = [{'type':'ineq', 'fun': lambda x: x[0]+2*x[1]-5},
        {'type':'ineq', 'fun': lambda x: x[0]},
        {'type':'ineq', 'fun': lambda x: x[1]}]
res = minimize(obj, [1,2], constraints=cons)
print(f"Optimal: {res.x}, Value: {res.fun}")

Optimal: [2. 3.], Value: 0.0
```

OUTPUT:

Optimal: [2. 3.], Value: 0.0

Problem 4

Aim: Maximize $f(x, y) = 4x + 3y$ subject to $x^2 + y^2 \leq 25$, $x, y \geq 0$.

Procedure: $\mathcal{L} = 4x + 3y + \lambda(25 - x^2 - y^2) + \mu_1 x + \mu_2 y$. Circle boundary active.

Coding:

```
[4]
✓ 0s
def obj(x): return -(4*x[0] + 3*x[1])
cons = [{'type':'ineq', 'fun': lambda x: 25-x[0]**2-x[1]**2},
        {'type':'ineq', 'fun': lambda x: x[0]},
        {'type':'ineq', 'fun': lambda x: x[1]}]
res = minimize(obj, [3,2], constraints=cons)
print(f"Maximum: {-res.fun}, Point: {res.x}")

Maximum: 25.000000000035087, Point: [4.00000002 2.99999997]
```

OUTPUT:

MAXIMUM: 2500000000000035087,point:[4.000000002 2.99999997]

Problem 5

Aim: Minimize $f(x,y) = \frac{x^2}{2} + \frac{y^2}{2}$ subject to $x + y \geq 1, 2x + y \leq 4, x, y \geq 0$.

Procedure: Multiple inequality constraints, check active sets via KKT.

Coding:

```
[5]
✓ 0s
def obj(x): return x[0]**2/2 + x[1]**2/2
cons = [{'type':'ineq', 'fun': lambda x: x[0]+x[1]-1},
        {'type':'ineq', 'fun': lambda x: 4-2*x[0]-x[1]},
        {'type':'ineq', 'fun': lambda x: x[0]},
        {'type':'ineq', 'fun': lambda x: x[1]}]
res = minimize(obj, [1,1], constraints=cons)
print(f"Optimal: {res.x}, Value: {res.fun}")

Optimal: [0.5 0.5], Value: 0.2499999999999999
```

Output:

Optimal: [0.5 0.5], Value: 0.2499999999999999

Problem 6

Aim: Maximize production $5x + 4y$ subject to $2x + 3y \leq 30$, $x \leq 10$, $x, y \geq 0$.

Procedure: Linear objective, verify KKT at corner point.

Coding:

```
[6]
✓ 0s
def obj(x): return -(5*x[0] + 4*x[1])
cons = [{'type':'ineq', 'fun': lambda x: 30-2*x[0]-3*x[1]},
        {'type':'ineq', 'fun': lambda x: 10-x[0]},
        {'type':'ineq', 'fun': lambda x: x[0]},
        {'type':'ineq', 'fun': lambda x: x[1]}]
res = minimize(obj, [5,5], constraints=cons)
print(f"Profit: {-res.fun}, Point: {res.x}")

Profit: 63.33333333332941, Point: [10.          3.33333333]
```

Output:

Profit: 63.33333333332941 ,point:[10. 3.33333333333]

Problem 7

Aim: Minimize $f(x, y) = e^x + e^y$ subject to $x + y = 3, x \geq 1$.

Procedure: Equality + inequality KKT conditions.

Coding:

```
import numpy as np
def obj(x): return np.exp(x[0]) + np.exp(x[1])
cons = [{'type':'eq', 'fun': lambda x: x[0]+x[1]-3},
        {'type':'ineq', 'fun': lambda x: x[0]-1}]
res = minimize(obj, [1.5,1.5], constraints=cons)
print(f"Minimum: {res.fun}, Point: {res.x}")
```

```
Minimum: 8.963378140676129, Point: [1.5 1.5]
```

Output:

Minimum: 8.963378140676129 Point: [1.5 1.5]